

Step 1 — Two Standalone MCP-Connected Specialists — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the real `mcp` SDK calls, in the right order), **Advanced** (what a real, working standalone specialist needs beyond the happy path). Read Basic first even if you already built Project 8 — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — Two servers, two agents, no sharing between them yet](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Two servers, two agents, no sharing between them yet

Basic Version

You're building 4 small pieces this step, and none of them know about each other yet: a notes server, a web server, a notes agent, and a web agent. Each server is its own separate Python file that runs as its own separate process. Each agent is its own separate Python file that starts one of those servers as a subprocess, asks it what it can do, and uses it.

Things to use: - `FastMCP` to build each server — one tool function per server for the web server, two for the notes server. - The same `stdio_client` / `ClientSession` / `initialize()` shape from Project 8's Step 1, once per agent. - A small in-memory Python list for the notes server's storage, and a small Python dict for the web server's mocked pages.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The two servers are both just `FastMCP` instances — the notes server is a smaller version of Project 7's `MCPForge` (no `SQLite`, no `Resource`, no `Prompt`, just the two `Tools`), and the web server is even smaller: one tool, backed by a plain dict lookup instead of a real network call.

The exact pieces for `notes_server.py` : - `mcp = FastMCP("notes-server")` - A module-level list, seeded with 3-4 strings at import time, so `search_notes` has real data before any tool is ever called. - `@mcp.tool() def add_note(text: str) -> str:` — appends to the list, returns a confirmation. - `@mcp.tool() def search_notes(query: str) -> list[str]:` — loops over the list, keeps any note where `query.lower()` is a substring of `note.lower()` . - `mcp.run(transport="stdio")` at the bottom.

The exact pieces for `web_server.py` : - `mcp = FastMCP("web-server")` - A module-level dict, `MOCK_PAGES = {"https://...": "..."} ,` with 2-3 entries. - `@mcp.tool() def fetch_page(url: str) -> str:` — looks up `url` in `MOCK_PAGES` , returns the canned text if found, or a clear `"No content available for this URL."` string if not. A comment right above this function should say plainly that this is a mocked stand-in for a real fetch/search MCP server.

For each agent (`notes_agent.py` , `web_agent.py`): reuse `mcp_connection.py` 's `connect_stdio_server` (the same shape as Project 8's `mcp_connection.py`), then reuse Project 8 Step 2's `mcp_tools_to_openai_schema` and a small hand-built ReAct loop — the agent doesn't need to be fancy, it just needs to actually call the model, let it pick a tool, execute the tool through the session, and answer.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about what "standalone, proven alone" actually needs to mean here, beyond just "the code runs once without crashing." Each agent needs its **own, separate** MCP connection — don't be tempted to open one connection and somehow share it between `notes_agent.py` and `web_agent.py`, even though that might look like less code. The whole point of this project is that each specialist owns its own connection to its own server; sharing a connection object between two unrelated specialists at this stage would quietly undermine Step 4's entire degradation story later, since a shared connection means one server's failure can't cleanly be isolated to just the specialist that depends on it.

Also think about what "seeded with real data" actually buys you here versus an empty notes list. If `search_notes` is tested against an empty list, a bug where it never matches anything correctly looks identical to a bug where the list itself is empty — seed the notes list with content that includes at least one note relevant to your Step 1 test task ("Project Atlas"), and one that isn't, so a search that returns the wrong note, or nothing at all, is actually visible as a bug instead of hidden by having nothing to find either way.

One more real thing to test now, even though it's small: call `run_notes_agent` twice in the same `main.py` run, back to back. Since each call opens and fully closes its own connection (`async` with `block` finishing at the end of the function), this should work cleanly both times — if it doesn't, that's a sign your connection isn't actually closing between calls, which will bite you the moment Step 2 calls a specialist's node function more than once in a longer graph run.

Things to try before Hint 2: - Run `search_notes` against a query that matches nothing on purpose, and confirm your agent's answer says so clearly instead of hallucinating a note that doesn't exist. - Call `fetch_page` with a URL that isn't in `MOCK_PAGES` and confirm the agent reports "no content" instead of inventing page content.

Difference between Basic, Intermediate, and Advanced: Basic names the 4 pieces and what each roughly needs. Intermediate gives the real `FastMCP` decorators and tool bodies for both servers, and points at Project 8's reusable agent-loop pieces. Advanced is about what "proven standalone" really requires — a genuinely separate connection per specialist, seeded test data specific enough to catch a wrong-match bug, and confirming the agent doesn't quietly invent an answer when its tool legitimately found nothing.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
notes_server.py:
    keep a list of notes, seeded with a few made-up ones
    tool add_note(text): append to the list, return confirmation
    tool search_notes(query): return notes containing query

web_server.py:
    keep a dict of url -> page text, a few made-up entries
    tool fetch_page(url): return the dict entry, or "no content" if missing

notes_agent.py / web_agent.py:
    connect to the matching server
    discover its tools
    run a small loop: ask the model, call whichever tool it picks, feed the result
    return the final answer

main.py:
    run_notes_agent(task about notes)
    run_web_agent(task about the web page)
    print both
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Here's almost the whole `notes_server.py` and `web_server.py` — type them out yourself and adjust the seed data to your own scenario:

```
# notes_server.py
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("notes-server")

_NOTES = [
    "Project Atlas kickoff meeting notes: launch targeted for Q3.",
    "Remember to buy milk and eggs.",
    "Project Atlas: waiting on legal sign-off before rollout.",
]

@mcp.tool()
def add_note(text: str) -> str:
    """Add a new note."""
    _NOTES.append(text)
    return f"Note added. There are now {len(_NOTES)} notes."

@mcp.tool()
def search_notes(query: str) -> list[str]:
    """Search notes for a substring, case-insensitive."""
    matches = []
    for note in _NOTES:
        if query.lower() in note.lower():
            matches.append(note)
    return matches

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

```
# web_server.py
# NOTE: fetch_page is mocked -- a real deployment would connect this tool
# to a real fetch/search MCP server (e.g. the public reference fetch server)
# instead of this hardcoded dict. Mocked here for reliable, repeatable teaching.
```

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("web-server")

 MOCK_PAGES = {
    "https://intranet.example.com/atlas/status": "Project Atlas status: on track,
}

@mcp.tool()
def fetch_page(url: str) -> str:
    """Fetch the text content of a known page (mocked)."""
    if url in _MOCK_PAGES:
        return _MOCK_PAGES[url]
    return "No content available for this URL."

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Your turn: write `run_notes_agent(task)` and `run_web_agent(task)` in their own files, reusing `mcp_connection.py` and the ReAct loop shape from Project 8's Step 2 `agent.py` .

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Fill in the agent loop yourself, using this skeleton for `notes_agent.py` (`web_agent.py` is the same shape, pointed at `web_server.py`):

```
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from tool_bridge import mcp_tools_to_openai_schema, call_mcp_tool # reused from

async def _run_notes_agent(task: str) -> str:
    server_params = StdioServerParameters(command="python", args=["notes_server.p
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools_result = await session.list_tools()
            openai_tools = mcp_tools_to_openai_schema(tools_result.tools)

            # your turn: build the message list, call the model, check for
            # tool calls, run any through call_mcp_tool(session, name, args),
            # append the result as a tool message, and repeat until the
            # model answers with no more tool calls (max_iterations, Doc07-style)
            ...

def run_notes_agent(task: str) -> str:
    """Sync entry point -- Step 2's graph node will call this directly."""
    return asyncio.run(_run_notes_agent(task))
```

Notice the split: `_run_notes_agent` is the real `async` logic, and `run_notes_agent` is a plain sync wrapper around `asyncio.run(...)`. Keep this split in both agents — it's exactly what lets Step 2's LangGraph node functions call a specialist without every node in the whole graph needing to be `async` itself.

Compare your finished loop and both servers against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic sketches both servers' data and both agents' loop in plain words. Intermediate gives the real, nearly-complete `FastMCP` server code for both. Advanced gives the sync/async split shape the agent functions need — not because Step 1

requires it yet, but because getting it right now saves you a rewrite in Step 2, when these same functions get called from inside a graph node.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)